МТУСИ

Московский технический университет
связи и информатики

ОТЧЁТ ПО ЛАБОРАТОРНОЙ РАБОТЕ №2

«Интеграция REST API с фронтендом на React»

Дисциплина: «Программная инженерия»

Вариант 5. Фильмотека «Кадр»

Выполнил: студент группы БСТ-2556
Новосельцев Алексей Сергеевич

Проверил: _____

Москва, 2026

1. Цель, задание и использованные средства

Цель - освоить взаимодействие React-приложения с REST API из лабораторной №1: получать данные по HTTP, отображать их и выполнять создание, изменение и удаление записей на клиенте.

Для варианта 5 разработана фильмотека «Кадр». Backend первой работы сохранён: FastAPI, SQLAlchemy и SQLite. В сервер добавлен CORSMiddleware. Данные отображаются карточками; запросы выполняются встроенным fetch.

Требование	Выполнение
Готовое API работы №1	Используется существующее приложение app/ и таблица movies
React + axios или fetch	React 19.3.1, fetch; сборка Vite 8.3.4
GET, POST, PUT, DELETE	Список и детали; добавление; замена; удаление
Не менее трёх экранов	Коллекция, форма, поиск, детали - четыре экрана
Поиск или фильтрация	Название, год, жанр, диапазон рейтинга
Корректный CORS	Прямые запросы между портами 5173 и 8000, preflight
Оформление	CSS, адаптивная сетка, SVG-постеры и состояния интерфейса
Документация	README, маршруты, примеры fetch, фактические скриншоты

2. Структура проекта

Файл	Назначение
frontend/src/main.jsx	React root, StrictMode, BrowserRouter
frontend/src/App.jsx	Навигация, коллекция, поиск, форма, детали
frontend/src/api.js	fetch, обработка HTTP-ошибок, JSON, валидация
frontend/src/components.jsx	Карточки, поля, загрузка, диалог, SVG-постеры
frontend/src/styles.css	Цвета, типографика, сетки и медиазапросы
frontend/tests/api.test.js	Проверки клиентской логики
app/main.py, tests/test_cors.py	Настройка и проверки CORS

3. Экраны приложения и маршруты API

Навигация реализована через React Router. Коллекция показывает данные API, статистику и быстрый поиск. Общая форма используется для добавления и редактирования. Поиск вынесен на отдельный экран; карточка фильма открывает все его поля.

Путь клиента	Экран / назначение
/	Коллекция: карточки, сортировка, страницы
/movies/new	Новая карточка, отправка POST
/movies/:id/edit	Заполненная форма, отправка PUT
/search	Название, год, жанр и диапазон рейтинга
/movies/:id	Детали, изменение и удаление

Маршруты React относятся к клиентскому интерфейсу. Они обращаются к следующим серверным операциям по адресу http://127.0.0.1:8000:

Метод	Путь API	Использование
GET	/api/v1/movies	Список и фильтры
GET	/api/v1/movies/{id}	Детали и исходные поля формы
POST	/api/v1/movies	Добавление, код 201
PUT	/api/v1/movies/{id}	Полная замена, код 200
DELETE	/api/v1/movies/{id}	Удаление, код 204 без JSON
GET	/api/v1/genres	Варианты жанра в поиске
GET	/api/v1/movies/stats	Число фильмов, среднее, жанры
GET	/health	Состояние соединения

Параметры q, year, genre, rating_min, rating_max, sort_by, order и page сохраняются в URL. На главной странице запрашивается до 8 фильмов, на странице поиска - до 6. Сортировка и фильтрация выполняются сервером, а не только над уже загруженными карточками.

При загрузке показан индикатор, при отсутствии совпадений - пояснение и сброс фильтров, при ошибке - сообщение и повтор запроса. Отсутствующий фильм обрабатывается как 404.

4. Взаимодействие через fetch и настройка CORS

Общий клиент request() проверяет response.ok, возвращает JSON успешного ответа и передаёт ошибки форме. Для DELETE с кодом 204 JSON не читается. Числа формы преобразуются через Number(), пустое описание - в null. Примеры:

```
const query = new URLSearchParams({
  q: 'ОРБИТА', year: '2024', rating_min: '8.5'
});
await request(`/api/v1/movies?${query}`);

const film = {
  title: 'Между кадрами', director: 'Анна Миронова',
  year: 2024, genre: 'Драма', rating: 8.7,
  duration_minutes: 115, description: null
};
const created = await request('/api/v1/movies', {
  method: 'POST', body: film
});
await request(`/api/v1/movies/${created.id}`, {
  method: 'PUT', body: { ...film, rating: 9.3 }
});
await request(`/api/v1/movies/${created.id}`, {
  method: 'DELETE'
});
```

GET-запросы запускаются через useEffect. При смене пути или фильтров AbortController отменяет прежний запрос. Обработчики событий выполняют POST, PUT и DELETE; форма блокируется до завершения сохранения [1, 4].

Frontend на порту 5173 и API на порту 8000 имеют разные origin. Браузер проверяет разрешение сервера через CORS. Для JSON-запросов он может сначала отправить OPTIONS preflight. В Vite proxy не используется, поэтому проверено именно междоменное обращение [3].

```
application.add_middleware(
    CORSMiddleware,
    allow_origins=[
        'http://127.0.0.1:5173', 'http://localhost:5173',
        'http://127.0.0.1:4173', 'http://localhost:4173',
    ],
    allow_credentials=False,
    allow_methods=['GET', 'POST', 'PUT', 'DELETE'],
    allow_headers=['Content-Type'],
    expose_headers=['Location'],
)
```

Разрешённые origin можно переопределить через CORS_ORIGINS. Адрес backend задаётся VITE_API_URL; значение по умолчанию - http://127.0.0.1:8000. При новом порту клиента нужно добавить его origin в серверную настройку.

5. Запуск и проверка приложения

Требуются Python 3.12+ и Node.js 22.12+. Сначала из корня репозитория запускается backend, затем в отдельном терминале - frontend [2].

```
# Терминал 1: корень проекта
python3 -m venv .venv
source .venv/bin/activate
python -m pip install -r requirements.txt
python -m app.seed
python -m uvicorn app.main:app --reload

# Терминал 2: корень проекта
npm install --global pnpm@11.25.0
cd frontend
pnpm install --frozen-lockfile
pnpm dev

# Проверка клиента и сборка
pnpm test
pnpm build
```

Интерфейс доступен по http://127.0.0.1:5173, Swagger UI - по http://127.0.0.1:8000/docs. В Windows окружение активируется командой .venv\Scripts\activate. pnpm preview показывает результат сборки на порту 4173.

Проверка	Фактический результат
Backend: pytest	73 теста пройдено, включая 7 проверок CORS
Покрытие backend без seed	191 из 192 строк, около 99%
Клиент: Node test runner	16 тестов пройдено: fetch, ошибки, JSON, валидация
Сборка Vite	pnpm build выполнен успешно
Проверка в браузере	13 сценариев выполнены вручную через интерфейс

В браузере создан фильм «Между кадрами» с id=9, рейтинг изменён с 8,7 на 9,3. Обновление страницы подтвердило сохранение. Отмена удаления оставила запись в коллекции; подтверждение удалило её, после чего детали показали 404. В исходной коллекции осталось 8 учебных фильмов.

Поиск ОРБИТА + 2024 + Фантастика + рейтинг 8.5..10 вернул только «Орбита памяти». Проверены пустая форма, неверный диапазон, переход на вторую страницу, сортировка и отсутствие совпадений. Результаты сохранены в docs/lab2/; клиентские unit-тесты не подменяют отдельную проверку интерфейса.

6. Главная страница: коллекция

Карточки получены из API. Сверху отображаются статистика, поиск, сортировка и переход к добавлению. Иллюстрации выполнены в SVG.

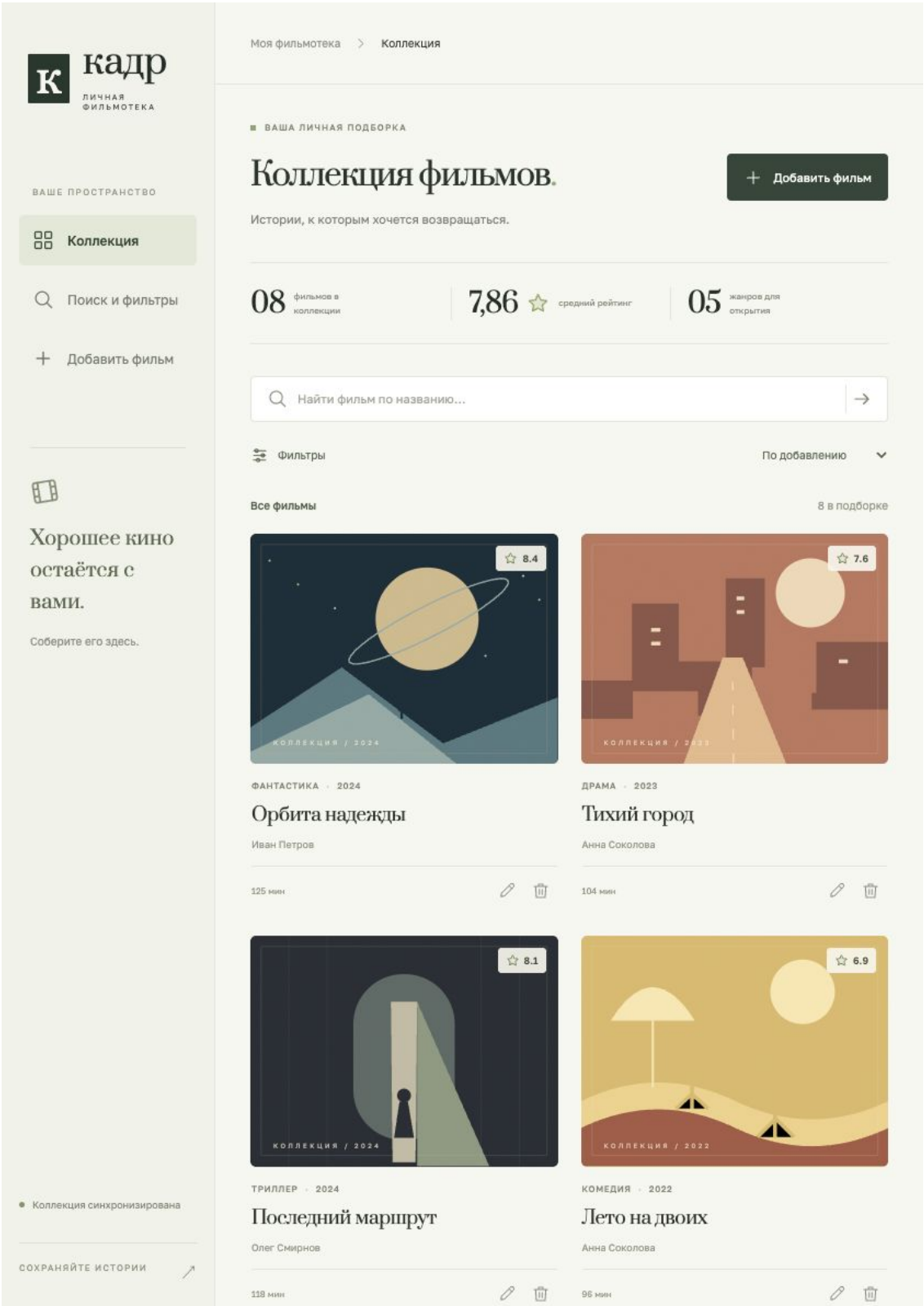


Рисунок 1 - список фильмов и статистика

7. Форма добавления и редактирования

На экране добавления введены данные контрольного фильма. Та же форма по /movies/:id/edit загружает существующие значения и отправляет PUT.

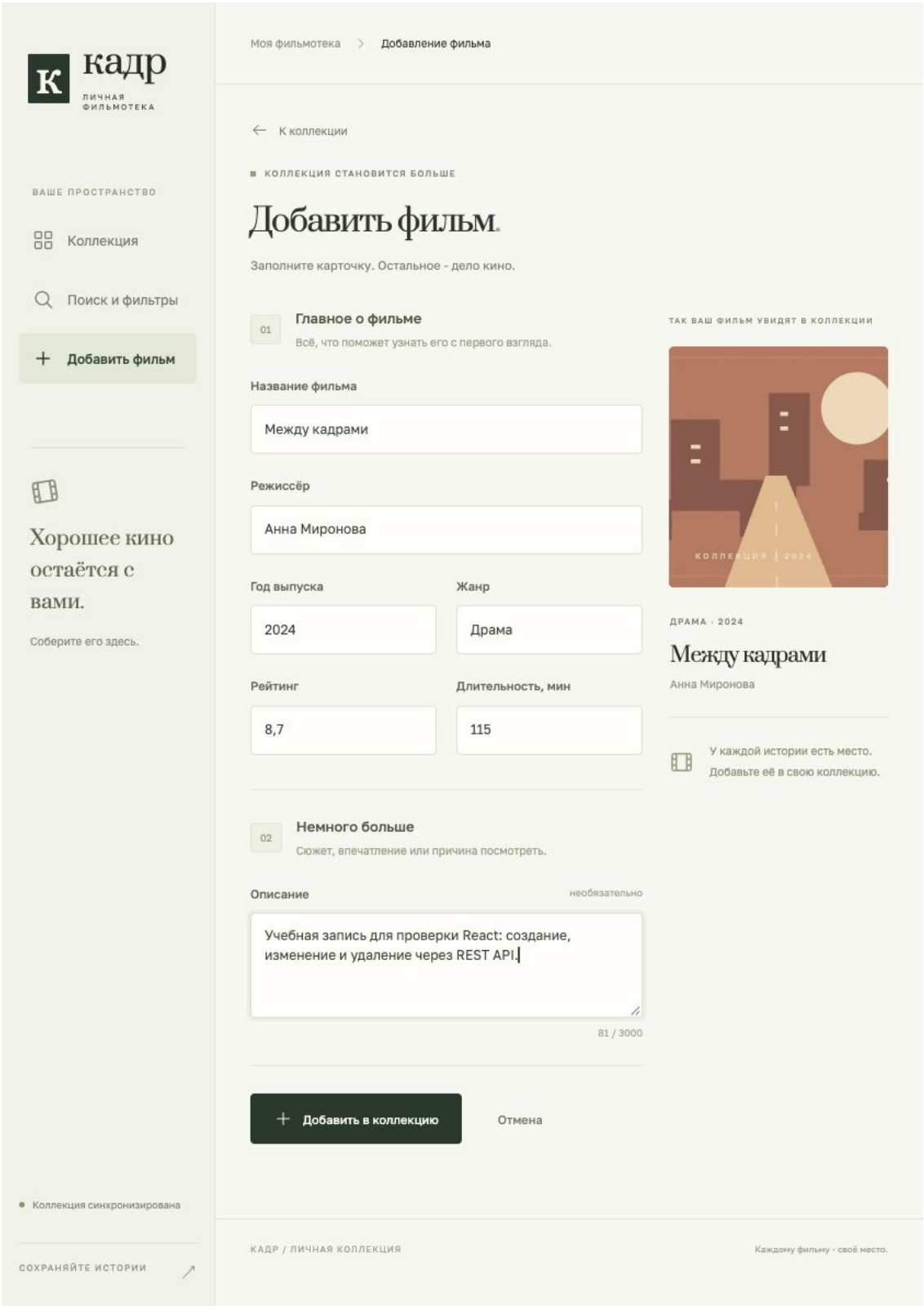


Рисунок 2 - заполненная форма добавления

8. Поиск и фильтрация

Одновременно применены поиск по названию, год, жанр и диапазон рейтинга. Сервер вернул один подходящий фильм; значения фильтров сохранены в URL.

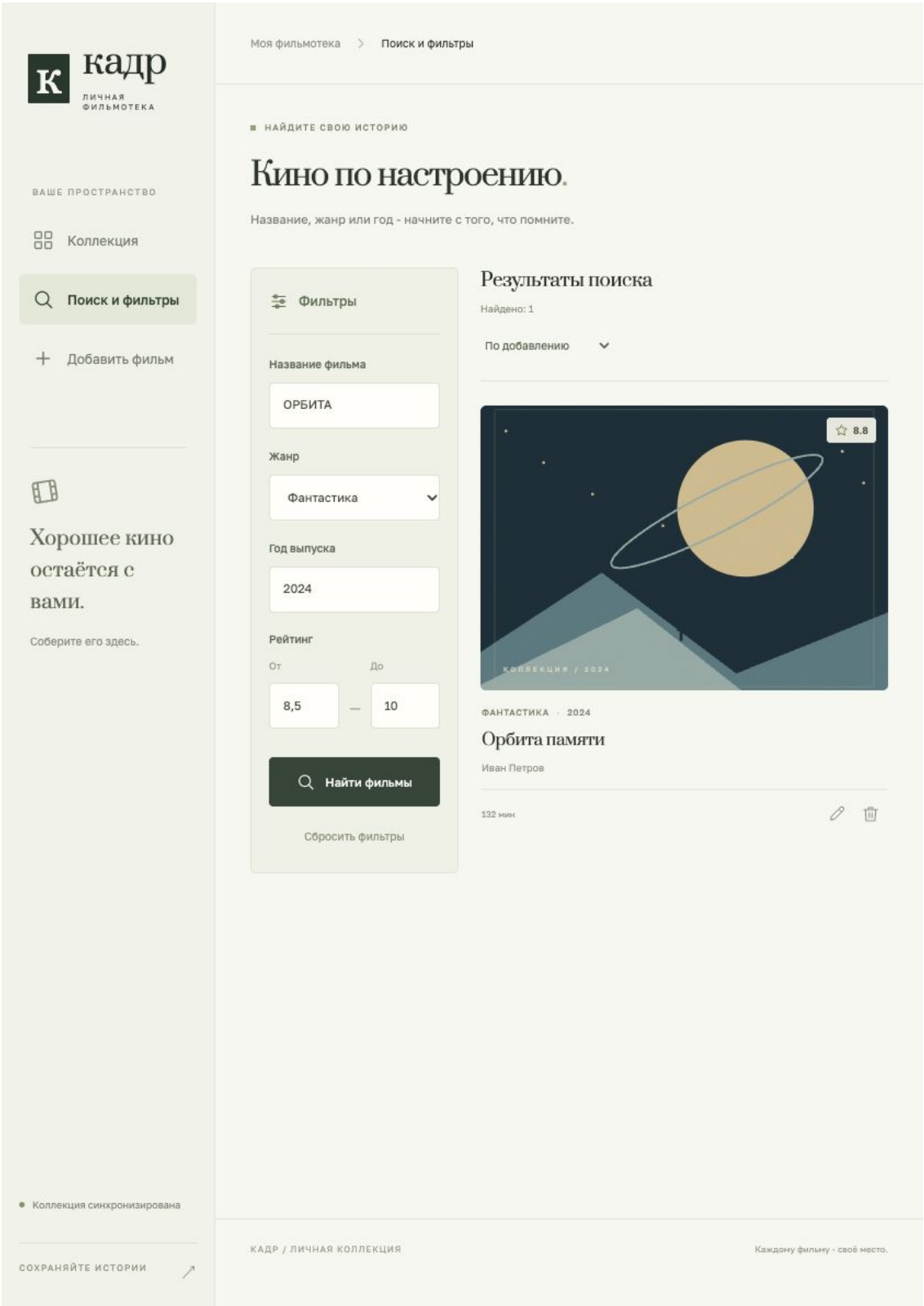


Рисунок 3 - результат комбинированного поиска

9. Страница деталей фильма

После POST открыт созданный фильм. Видны все поля, рейтинг и действия редактирования и удаления. Контрольная запись после проверки удалена.

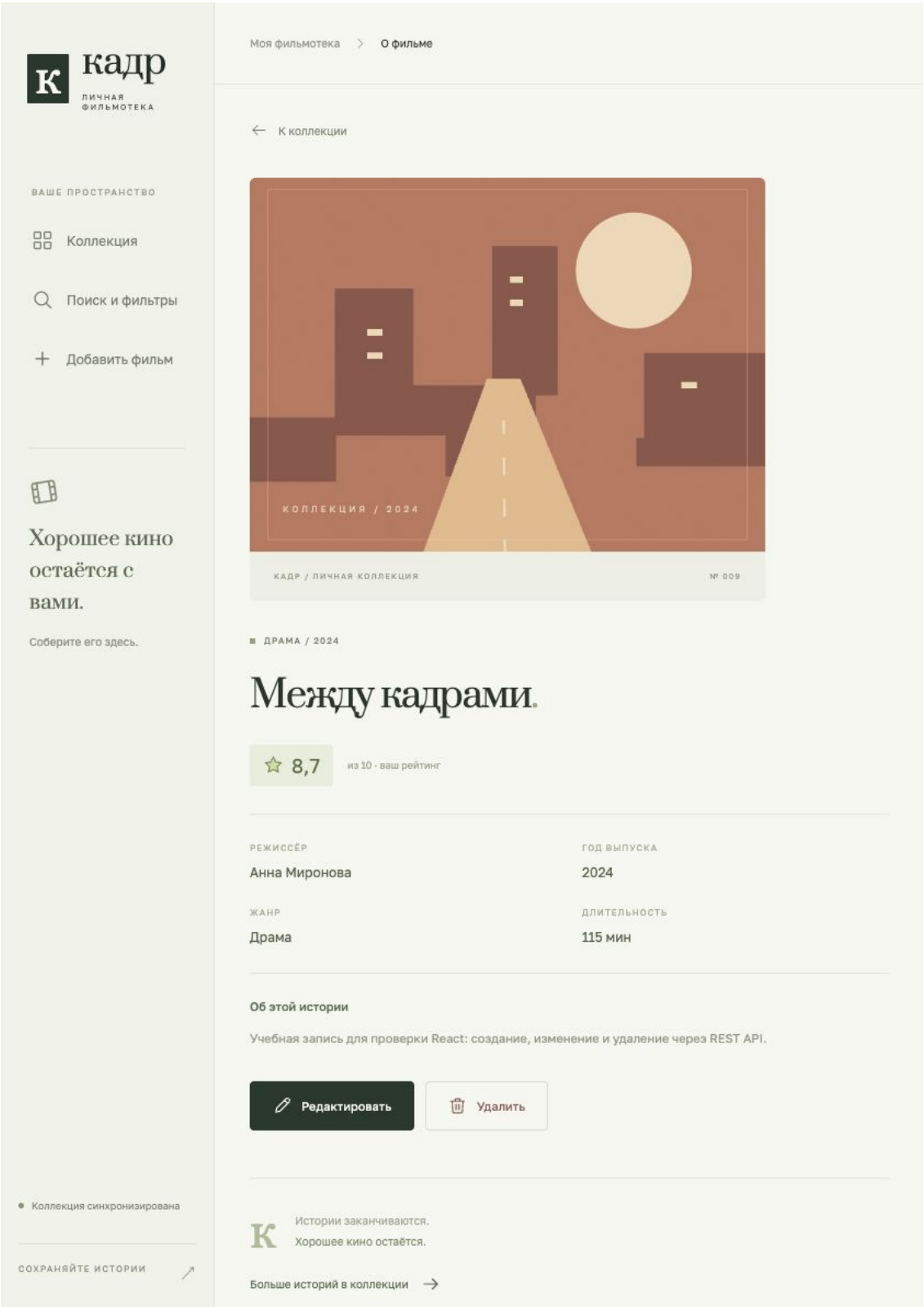


Рисунок 4 - подробная карточка фильма

10. Вывод

Выполнена интеграция React-приложения с REST API фильмотеки из первой лабораторной работы. Реализованы получение и отображение данных, добавление, редактирование, подтверждаемое удаление, поиск и фильтрация. Минимальное требование к числу экранов выполнено: доступны коллекция, общая форма, поиск и детали.

Клиент использует fetch, обрабатывает ошибки и состояния загрузки. Сервер разрешает обращения с origin frontend через CORSMiddleware; GET и изменяющие операции проверены при работе двух локальных серверов. Изменения сохраняются в SQLite и доступны после обновления страницы.

Тесты backend и клиентской логики пройдены, сборка выполнена. Скриншоты получены из работающего приложения. Цель работы достигнута: освоено взаимодействие браузерного интерфейса с серверным REST API.

11. Комплект сдачи

Исходный код находится в app/ и frontend/. Краткая документация со структурой, API-маршрутами, fetch-примерами и скриншотами - frontend/README.md. Проверки и иллюстрации - docs/lab2/. Отчёт второй работы сохранён отдельно: output/pdf/lab2-report.pdf.

Репозиторий: github.com/Ersan1908/prog_e.

12. Источники

1. React. Synchronizing with Effects.
https://react.dev/learn/synchronizing-with-effects

2. Vite. Getting Started.
https://vite.dev/guide/

3. FastAPI. CORS.
https://fastapi.tiangolo.com/tutorial/cors/

4. MDN. Using the Fetch API.
https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API/Using_Fetch

Дата обращения: 21.10.2026.